

■ JARVIS OS

The Complete 365-Day Roadmap

Building a Production Windows AI Agent

18 Modules	6 Phases	365 Daily Lessons	50+ Books	100+ Research Papers	300+ GitHub Repos
------------	----------	-------------------	-----------	----------------------	-------------------

From writing your first Python decorator to deploying a production-grade AI agent that controls every application on Windows — one day at a time.

Target: AI comparable to Cursor Agent · OpenHands · Manus · Devin

Table of Contents

Phase 1: Programming Foundations (1–30)	Days 1–30
Week 1: Python Mastery	1–7
Week 2: Data Structures & Algorithms	8–14
Week 3: Git, Linux & Networking	15–21
Week 4: Databases & System Design	22–30
Phase 2: Windows Internals & Systems Programming (31–60)	Days 31–60
Week 5: Windows Architecture & Processes	31–37
Week 6: Win32 API & UI Automation	38–44
Week 7: Rust Foundations	45–51
Week 8: Desktop Automation Deep Dive	52–60
Phase 3: AI & LLM Engineering (61–120)	Days 61–120
Week 9: LLM Fundamentals	61–67
Week 10: RAG & Knowledge Systems	68–74
Week 11: LangChain & LangGraph	75–81
Week 12: Agent Architectures & Memory	82–90
Phase 4: Browser Automation & Computer Vision (121–180)	Days 121–180
Week 13: Playwright & Browser Control	121–127
Week 14: Data Extraction & Web Intelligence	128–134
Week 15: Computer Vision – Deep Dive	135–141
Week 16: Integration – Browser + Vision + LLM	142–150
Phase 5: Multi-Agent Systems & Production (181–270)	Days 181–270
Week 17: Multi-Agent Orchestration	181–187
Week 18: Windows Agent + Terminal Agent + Code Agent	188–210
Phase 6: Production, Polish & Capstone (271–365)	Days 271–365
Week 19: Production Engineering	271–300
Week 20: Final Capstone – Jarvis OS	293–365

How to Use This Roadmap

This roadmap assumes **3–6 hours of daily study and practice**. Each day has five components:

■ Study	Books, official docs, or blog posts to read
■ Watch	YouTube video or lecture to watch (30–60 min)
■ Repo	GitHub repository to clone, explore, and run
■ ■ Exercise	Hands-on coding exercise — always build something
■ Outcome	What you should be able to do by end of day

Weekly Projects: Every week ends with a capstone mini-project that integrates everything from that week. **Monthly Capstones:** Days 30, 60, 90, 150, 210, and 365 are full-day integration projects. **Research Papers:** Read them on arXiv (arxiv.org) — search by paper title. **YouTube:** All videos are freely available — search by channel name + video title. **GitHub Repos:** Clone and run locally — reading source code is as important as reading books.

Phase 1 — ■■ Programming Foundations

Days 1–30 · Goal: Build rock-solid Python, Git, and Linux foundations before touching AI or agents.

Week 1 | Python Mastery | Days 1–7

Day 1 | Python Environment + Data Types

- Study Python Docs – Built-in Types; Fluent Python Ch. 1–2
- Watch Corey Schafer – Python Tutorial for Beginners (YT)
- Repo github.com/TheAlgorithms/Python
- Exercise Write 20 functions covering list, dict, set, tuple. Include slicing, comprehensions, unpacking.
- Outcome Comfortable with all Python primitives

Day 2 | Functions, Closures & Decorators

- Study Fluent Python Ch. 5–7; Real Python – Decorators
- Watch ArjanCodes – Decorators Deep Dive (YT)
- Repo github.com/dabeaz-course/python-mastery
- Exercise Build a retry decorator, a timer decorator, and a memoize decorator from scratch.
- Outcome Can explain closures, first-class functions, and decorator chaining

Day 3 | OOP – Classes, Inheritance, Protocols

- Study Fluent Python Ch. 11–13; Python Docs – data model
- Watch mCoding – Python OOP (YT)
- Repo github.com/faif/python-patterns
- Exercise Implement a plugin system using `__init_subclass__` and a registry pattern.
- Outcome Mastery of dunder methods, MRO, and abstract base classes

Day 4 | Generators, Iterators & Context Managers

- Study Fluent Python Ch. 14–15; PEP 342, PEP 380
- Watch James Powell – Generators (YT – PyCon)
- Repo github.com/pytoolz/toolz
- Exercise Build a lazy file reader pipeline that processes 1 GB log file with O(1) memory.
- Outcome Deep understanding of iteration protocol and yield from

Day 5

Asyncio – Event Loop, Coroutines

- **Study** Python Docs – asyncio; Real Python – Async IO
- **Watch** mCoding – asyncio event loop deep dive (YT)
- **Repo** github.com/python/cpython/tree/main/Lib/asyncio
- **Exercise** Build an async HTTP client that fetches 100 URLs concurrently using aiohttp.
- **Outcome** Understand event loop, tasks, gather, and semaphores

Day 6

Threading, Multiprocessing & Subprocess

- **Study** Fluent Python Ch. 17–19; Python Docs – concurrent.futures
- **Watch** Tech With Tim – Python Multithreading (YT)
- **Repo** github.com/bslatkin/effectivepython
- **Exercise** Benchmark ThreadPoolExecutor vs ProcessPoolExecutor for CPU-bound vs I/O-bound tasks.
- **Outcome** Know when to use threads vs processes vs async

Day 7

Error Handling, Logging & Testing

- **Study** Python Docs – exceptions; Real Python – pytest
- **Watch** Pytest Full Course – Telusko (YT)
- **Repo** github.com/pytest-dev/pytest
- **Exercise** Add full test suite + logging to your Week 1 exercises. Target 90% coverage.
- **Outcome** Professional error handling and test-driven workflow

- **Weekly Project** Build an async web scraper that fetches data from 50 sites concurrently, stores results in SQLite, and has full test coverage.

Week 2 | Data Structures & Algorithms | Days 8–14

Day 8

Arrays, Strings & Complexity

- **Study** CTCI Ch. 1; NeetCode roadmap – Arrays
- **Watch** NeetCode – Array Problems (YT)
- **Repo** github.com/neetcode-gh/leetcode
- **Exercise** Solve 10 LeetCode array/string problems (Easy–Medium)
- **Outcome** O(n) intuition for common patterns

Day 9	Linked Lists, Stacks & Queues
■ Study	CTCI Ch. 2–3; Python Docs – collections.deque
■ Watch	Abdul Bari – Linked List (YT)
■ Repo	github.com/TheAlgorithms/Python
■ Exercise	Implement singly + doubly linked list, LRU cache
■ Outcome	Pointer manipulation confidence
Day 10	Trees & Graphs – BFS, DFS
■ Study	CTCI Ch. 4; Introduction to Algorithms Ch. 22–23
■ Watch	William Fiset – Graph Theory (YT)
■ Repo	github.com/williamfiset/algorithms
■ Exercise	Implement BFS/DFS on adjacency list, detect cycles, topological sort
■ Outcome	Graph traversal mastery
Day 11	Heaps, Priority Queues & Hash Maps
■ Study	Introduction to Algorithms Ch. 6; Python heapq docs
■ Watch	NeetCode – Heap problems (YT)
■ Repo	github.com/neetcode-gh/leetcode
■ Exercise	Implement min-heap from scratch; solve 5 top-K problems
■ Outcome	Know when and how to use heaps
Day 12	Dynamic Programming – Part 1
■ Study	Introduction to Algorithms Ch. 15; CTCI Ch. 8
■ Watch	Tushar Roy – DP playlist (YT)
■ Repo	github.com/neetcode-gh/leetcode
■ Exercise	Fibonacci, knapsack, LCS, coin change
■ Outcome	Memoization and tabulation patterns
Day 13	Dynamic Programming – Part 2 + Backtracking
■ Study	NeetCode DP 2D patterns; CTCI recursive problems
■ Watch	Back To Back SWE – DP (YT)
■ Repo	github.com/neetcode-gh/leetcode
■ Exercise	Word break, subset sum, N-Queens, permutations
■ Outcome	2D DP and decision-tree backtracking

Day 14

Sorting Algorithms & Binary Search

- **Study** Introduction to Algorithms Ch. 7–9; Python docs – bisect
- **Watch** Abdul Bari – Sorting algorithms (YT)
- **Repo** github.com/TheAlgorithms/Python
- **Exercise** Implement quicksort, mergesort, heapsort from scratch; 5 binary search problems
- **Outcome** Sort internals + $\log(n)$ search patterns

- **Weekly Project** Implement a mini search engine with inverted index, TF-IDF ranking, and query parsing using your DSA knowledge.

Week 3 | Git, Linux & Networking | Days 15–21

Day 15

Git – Branching, Rebasing, Internals

- **Study** Pro Git (free online) Ch. 1–4; Atlassian Git tutorials
- **Watch** Fireship – Git in 100 Seconds + advanced (YT)
- **Repo** github.com/git/git
- **Exercise** Create a repo, practice rebase vs merge, cherry-pick, bisect, stash
- **Outcome** Confident with any Git workflow

Day 16

Linux – File System, Processes, Signals

- **Study** The Linux Command Line (TLCL) Ch. 1–10
- **Watch** DorianDotSlash – Linux commands (YT)
- **Repo** github.com/torvalds/linux (browse)
- **Exercise** Navigate FS, manage processes with ps/kill/nice, write bash scripts
- **Outcome** Linux command-line proficiency

Day 17

Linux – Shell Scripting & Cron

- **Study** TLCL Ch. 24–31; Bash Guide (mywiki.woledge.org)
- **Watch** NetworkChuck – Bash Scripting (YT)
- **Repo** github.com/dylanaraps/pure-bash-bible
- **Exercise** Write bash scripts: file watcher, log rotator, backup tool
- **Outcome** Automate system tasks with bash

Day 18

Networking – TCP/IP, HTTP, DNS

- **Study** Computer Networking: A Top-Down Approach Ch. 1–2; MDN HTTP docs
- **Watch** Hussein Nasser – How HTTP works (YT)
- **Repo** github.com/trimstray/the-book-of-secret-knowledge
- **Exercise** Use curl, nc, Wireshark; implement TCP echo server in Python
- **Outcome** Understand packets, sockets, and protocols

Day 19

REST APIs, WebSockets & gRPC

- **Study** RESTful API Design – Best Practices; gRPC docs
- **Watch** TechWorld with Nana – REST vs gRPC (YT)
- **Repo** github.com/grpc/grpc
- **Exercise** Build a FastAPI REST server + WebSocket endpoint; consume with Python client
- **Outcome** Design and call APIs over all major protocols

Day 20

Docker – Containers & Images

- **Study** Docker Official Docs – Get Started; Docker Deep Dive (Nigel Poulton)
- **Watch** TechWorld with Nana – Docker Tutorial (YT)
- **Repo** github.com/docker/compose
- **Exercise** Containerize your web scraper from Week 1; multi-stage build
- **Outcome** Build, run, push Docker images

Day 21

Software Architecture Patterns

- **Study** Clean Architecture – Robert C. Martin Ch. 1–5; SOLID principles
- **Watch** ArjanCodes – Clean Architecture Python (YT)
- **Repo** github.com/faif/python-patterns
- **Exercise** Refactor Week 1 scraper to use repository pattern + dependency injection
- **Outcome** Understand layered architecture

- **Weekly Project** Build a Dockerised REST API with authentication, rate limiting, logging, and full CI pipeline using GitHub Actions.

Week 4 | Databases & System Design | Days 22–30

Day 22	SQL – PostgreSQL Deep Dive
■ Study	PostgreSQL Docs – Tutorial; Use The Index Luke (use-the-index-luke.com)
■ Watch	Hussein Nasser – PostgreSQL internals (YT)
■ Repo	github.com/dhamaniasad/awesome-postgres
■ Exercise	Design schema for a task management system; write complex JOINS, CTEs, window functions
■ Outcome	Write optimised SQL queries
Day 23	ORMs – SQLAlchemy & Migrations
■ Study	SQLAlchemy 2.0 Docs – ORM tutorial
■ Watch	Patrick Loeber – SQLAlchemy (YT)
■ Repo	github.com/sqlalchemy/sqlalchemy
■ Exercise	Port yesterday's schema to SQLAlchemy + Alembic migrations
■ Outcome	Database-agnostic Python data layer
Day 24	Redis – Caching, Pub/Sub, Queues
■ Study	Redis Docs – Commands; Redis in Action (free chapters)
■ Watch	ByteByteGo – Redis (YT)
■ Repo	github.com/redis/redis-py
■ Exercise	Add Redis caching + pub/sub messaging to your API from Week 3
■ Outcome	Use Redis for caching and event streaming
Day 25	Vector Databases – Qdrant & Embeddings
■ Study	Qdrant Docs – Quickstart; Pinecone Learning Center
■ Watch	AssemblyAI – Vector databases explained (YT)
■ Repo	github.com/qdrant/qdrant
■ Exercise	Store 1000 text embeddings in Qdrant; implement semantic search
■ Outcome	Understand vector similarity search
■ Paper	Approximate Nearest Neighbors Oh My (Spotify)
Day 26	System Design – Fundamentals
■ Study	Designing Data-Intensive Applications (DDIA) Ch. 1–3
■ Watch	ByteByteGo – System Design (YT playlist)
■ Repo	github.com/donnemartin/system-design-primer
■ Exercise	Design a URL shortener on paper; estimate QPS, storage, bandwidth
■ Outcome	Back-of-envelope estimation and component selection

Day 27

Message Queues – Kafka & RabbitMQ

- **Study** Kafka Docs – Introduction; DDIA Ch. 11
- **Watch** Amigoscode – Kafka Tutorial (YT)
- **Repo** github.com/confluentinc/confluent-kafka-python
- **Exercise** Set up Kafka locally; producer sends task messages, consumer executes them
- **Outcome** Async event-driven architecture basics
- **Paper** Kafka: a Distributed Messaging System (LinkedIn)

Day 28

Observability – Logs, Traces & Metrics

- **Study** OpenTelemetry Docs; Grafana + Prometheus Quickstart
- **Watch** TechWorld with Nana – Prometheus & Grafana (YT)
- **Repo** github.com/open-telemetry/opentelemetry-python
- **Exercise** Instrument your API with OpenTelemetry; view traces in Jaeger
- **Outcome** Add production-grade observability to any service

Day 29

Testing Strategies – Unit, Integration, E2E

- **Study** Python Testing with pytest (Brian Okken); Hypothesis docs
- **Watch** mCoding – Python testing (YT)
- **Repo** github.com/HypothesisWorks/hypothesis
- **Exercise** Write property-based tests for your DSA implementations using Hypothesis
- **Outcome** Multi-layer testing confidence

Day 30

Month 1 Capstone – Mini Backend System

- **Study** Review all Month 1 topics
- **Watch** —
- **Repo** —
- **Exercise** Build a full task-queue system: FastAPI → Kafka → Worker → PostgreSQL → Redis cache → Prometheus metrics → Docker Compose
- **Outcome** End-to-end backend system running locally

- **Weekly Project** Full capstone – see Day 30 above.

Phase 2 — ■ Windows Internals & Systems Programming

Days 31–60 · Goal: Understand how Windows works at the OS level — processes, memory, Win32, UI Automation. This is what separates real desktop agent builders from everyone else.

Week 5 | Windows Architecture & Processes | Days 31–37

Day 31 | Windows Architecture Overview

- **Study** Windows Internals Part 1 Ch. 1–2 (Mark Russinovich)
- **Watch** Pavel Yosifovich – Windows Internals (Pluralsight / YT)
- **Repo** github.com/zodiacon/windowsinternalbook
- ■ **Exercise** Use Sysinternals Process Explorer, Process Monitor, and Autoruns. Document your findings.
- **Outcome** Mental model of Windows components

Day 32 | Processes & Threads

- **Study** Windows Internals Part 1 Ch. 4
- **Watch** Dave Plummer – Task Manager internals (YT)
- **Repo** github.com/processhacker/processhacker
- ■ **Exercise** Write C++ program that creates child processes, injects a DLL (study only), enumerates threads
- **Outcome** Process/thread creation and management via Win32

Day 33 | Memory Management & Virtual Memory

- **Study** Windows Internals Part 1 Ch. 5
- **Watch** Low Level Learning – Virtual Memory (YT)
- **Repo** github.com/microsoft/WinObjC
- ■ **Exercise** Use VirtualAlloc, ReadProcessMemory, WriteProcessMemory in C++
- **Outcome** Understand pages, heaps, and address spaces

Day 34 | Registry & Configuration

- **Study** Windows Internals Part 2 Ch. 10; MSDN Registry docs
- **Watch** NetworkChuck – Registry deep dive (YT)
- **Repo** github.com/mkey/registry
- ■ **Exercise** Write Python script using winreg to read/write registry; enumerate startup entries
- **Outcome** Registry manipulation for automation

Day 35

Windows Services & Task Scheduler

- **Study** MSDN – Windows Services; Task Scheduler API docs
- **Watch** David Bombal – Windows Services (YT)
- **Repo** github.com/winsw/winsw
- **Exercise** Create a Python Windows Service using pywin32; schedule it with Task Scheduler
- **Outcome** Background process management on Windows

Day 36

COM & OLE Automation

- **Study** Programming Windows (Petzold) Ch. 20; MSDN COM docs
- **Watch** Jacob Salmela – COM Automation in Python (YT)
- **Repo** github.com/mhammond/pywin32
- **Exercise** Control Excel, Word, Outlook via COM using win32com.client in Python
- **Outcome** Automate any COM-enabled application

Day 37

PowerShell Scripting

- **Study** Learn PowerShell in a Month of Lunches (Don Jones) Ch. 1–10
- **Watch** NetworkChuck – PowerShell Tutorial (YT)
- **Repo** github.com/PowerShell/PowerShell
- **Exercise** Write PS scripts: system info harvester, file search tool, process manager
- **Outcome** PowerShell as automation backbone

- **Weekly Project** Build a Windows system info dashboard in Python using pywin32 + WMI: lists processes, services, startup items, registry keys, with a web UI.

Week 6 | Win32 API & UI Automation | Days 38–44

Day 38

Win32 API – Windows, Messages, WndProc

- **Study** Programming Windows (Petzold) Ch. 1–5; MSDN Win32 API
- **Watch** Pavel Yosifovich – Win32 Programming (YT)
- **Repo** github.com/microsoft/Windows-classic-samples
- **Exercise** Create a Win32 window in C++ that responds to keyboard and mouse messages
- **Outcome** Understand the Windows message loop

Day 39	Window Enumeration & Handle Manipulation
■ Study	MSDN – EnumWindows, FindWindow, GetWindowText
■ Watch	Low Level Learning – Window handles (YT)
■ Repo	github.com/zephraph/windows-hacking
■ Exercise	Write Python script using ctypes to enumerate all windows, get titles, classes, PIDs
■ Outcome	Find and interact with any window programmatically
Day 40	Accessibility API – MSAA & UIA
■ Study	MSDN – UI Automation overview; IAccessible docs
■ Watch	Jared Parsons – Accessibility API (CH9)
■ Repo	github.com/FlaUI/FlaUI
■ Exercise	Use Python uiautomation library to inspect any app's element tree; click buttons by name
■ Outcome	Reliable UI element detection without screenshots
Day 41	Keyboard & Mouse Hooks
■ Study	MSDN – SetWindowsHookEx; pywin32 hooks docs
■ Watch	Tech With Tim – Python keyboard hooks (YT)
■ Repo	github.com/moses-palmer/pynput
■ Exercise	Implement a global keyboard recorder and mouse tracker using low-level hooks
■ Outcome	Capture system-wide input events
Day 42	Clipboard & Drag-and-Drop
■ Study	MSDN – Clipboard, OLE Data Transfer
■ Watch	Adam Sporka – Clipboard programming (YT)
■ Repo	github.com/asweigart/pyperclip
■ Exercise	Build clipboard history manager; implement drag-drop file acceptor
■ Outcome	Clipboard integration for automation
Day 43	WMI – System Information & Control
■ Study	WMI Docs – Win32 Classes reference; wmi Python docs
■ Watch	Jeremy Morgan – WMI Python (YT)
■ Repo	github.com/tjuguk/wmi
■ Exercise	Query CPU, RAM, disk, network via WMI; monitor process creation/termination events
■ Outcome	Real-time system monitoring via WMI

Day 44

pyautogui & Screen Control

- **Study** pyautogui docs; pywinauto docs
- **Watch** Tech With Tim – pyautogui tutorial (YT)
- **Repo** github.com/asweigart/pyautogui
- **Exercise** Write script to open Notepad, type text, save file, close – using pyautogui and pywinauto
- **Outcome** Pixel-level + accessibility-level screen control

- **Weekly Project** Build a Windows Desktop Spy tool: given any app name, enumerate its entire UI tree, take a screenshot, highlight all interactive elements, and output a JSON map of all controls.

Week 7 | Rust Foundations | Days 45–51

Day 45

Rust – Ownership & Borrowing

- **Study** The Rust Programming Language (TRPL) Ch. 1–4 (book.rust-lang.org)
- **Watch** No Boilerplate – Rust ownership (YT)
- **Repo** github.com/rust-lang/rustlings
- **Exercise** Complete rustlings exercises 1–30
- **Outcome** Own/borrow mental model solid

Day 46

Rust – Structs, Enums, Pattern Matching

- **Study** TRPL Ch. 5–6, 18
- **Watch** Traversy Media – Rust crash course (YT)
- **Repo** github.com/rust-lang/rustlings
- **Exercise** Implement a state machine in Rust using enums + match
- **Outcome** Idiomatic Rust data modeling

Day 47

Rust – Traits, Generics & Lifetimes

- **Study** TRPL Ch. 10; Rust Reference – Traits
- **Watch** Jon Gjengset – Rust generics (YT)
- **Repo** github.com/dtolnay/rust-quiz
- **Exercise** Implement a generic stack and queue with trait bounds
- **Outcome** Polymorphism the Rust way

Day 48

Rust – Error Handling & Result

- **Study** TRPL Ch. 9; thiserror + anyhow docs
- **Watch** No Boilerplate – Error handling (YT)
- **Repo** github.com/dtolnay/anyhow
- **Exercise** Build a file processing CLI with proper error propagation using ?
- **Outcome** Production Rust error handling

Day 49

Rust – Async & Tokio

- **Study** TRPL Ch. 17 (async); tokio.rs tutorial
- **Watch** Jon Gjengset – Async Rust (YT)
- **Repo** github.com/tokio-rs/tokio
- **Exercise** Build an async TCP echo server with tokio; handle 100 concurrent connections
- **Outcome** Async Rust runtime usage

Day 50

Rust – FFI & Calling Win32

- **Study** TRPL Ch. 19 (unsafe); windows-rs crate docs
- **Watch** Ryan Levick – Rust Windows (YT)
- **Repo** github.com/microsoft/windows-rs
- **Exercise** Call EnumWindows Win32 API from Rust using windows-rs; print all window titles
- **Outcome** Call any Windows API from Rust

Day 51

Tauri – Desktop App Framework

- **Study** Tauri v2 Docs – Quickstart, Commands, Events
- **Watch** Fireship – Tauri 2 (YT)
- **Repo** github.com/tauri-apps/tauri
- **Exercise** Create a Tauri app with React frontend + Rust backend command; pass data both ways
- **Outcome** Full-stack desktop app foundation

- **Weekly Project** Build a Tauri desktop app that shows real-time system metrics (CPU, RAM, processes) fetched via Rust + windows-rs, displayed in a React dashboard.

Week 8 | Desktop Automation Deep Dive | Days 52–60

Day 52	pywinauto – Control Any App
■ Study	pywinauto docs; pywinauto GitHub wiki
■ Watch	Automation Step By Step – pywinauto (YT)
■ Repo	github.com/pywinauto/pywinauto
■ Exercise	Fully automate Calculator: open, perform 10 calculations, verify results
■ Outcome	Reliable Win32/UIA app control
Day 53	AutoHotKey – Scripting & Hotkeys
■ Study	AutoHotKey v2 Docs – Getting Started
■ Watch	The AHK Guy – AHK full course (YT)
■ Repo	github.com/AutoHotkey/AutoHotkey
■ Exercise	Write AHK scripts for window management, text expansion, and app launching
■ Outcome	AHK as a fallback automation layer
Day 54	WinAppDriver – WebDriver Protocol for Desktop
■ Study	WinAppDriver Docs; Appium Desktop docs
■ Watch	Selenium Conf – WinAppDriver talk (YT)
■ Repo	github.com/microsoft/WinAppDriver
■ Exercise	Write Python tests for Calculator and Notepad using WinAppDriver + selenium-python
■ Outcome	WebDriver-style desktop test automation
Day 55	OpenCV – Screen Analysis
■ Study	OpenCV Python Docs – Core Operations; Template Matching
■ Watch	sentdex – OpenCV Python (YT)
■ Repo	github.com/opencv/opencv-python
■ Exercise	Find a button icon on screen using template matching; click it with pyautogui
■ Outcome	Vision-based UI element detection
Day 56	OCR – EasyOCR & PaddleOCR
■ Study	EasyOCR Docs; PaddleOCR Docs
■ Watch	Nicholas Renotte – OCR in Python (YT)
■ Repo	github.com/PaddlePaddle/PaddleOCR
■ Exercise	Take a screenshot, run OCR, extract all text with bounding boxes, store in JSON
■ Outcome	Extract text from any screen region

Day 57**Combining UIA + CV + OCR**

- **Study** Study OmniParser paper and repo
- **Watch** Microsoft Research – OmniParser demo (YT)
- **Repo** github.com/microsoft/OmniParser
- **Exercise** Build a screen parser that returns structured JSON: {element_type, label, bbox, text} for every UI element visible on screen
- **Outcome** Complete screen understanding pipeline
- **Paper** OmniParser for Pure Vision Based GUI Agent (Microsoft)

Day 58**File System Automation**

- **Study** Python Docs – pathlib, shutil, watchdog
- **Watch** Corey Schafer – File automation (YT)
- **Repo** github.com/gorakhargosh/watchdog
- **Exercise** Build a file organiser that watches ~/Downloads and auto-sorts files by type
- **Outcome** Reliable cross-platform file operations

Day 59**Subprocess & Terminal Control**

- **Study** Python Docs – subprocess; pexpect docs
- **Watch** Corey Schafer – Subprocess module (YT)
- **Repo** github.com/pexpect/pexpect
- **Exercise** Build a terminal wrapper that captures output, injects input, and monitors for patterns
- **Outcome** Programmatic terminal session control

Day 60**Month 2 Capstone – Windows Automation Agent**

- **Study** Review all Month 2 topics
- **Watch** —
- **Repo** —
- **Exercise** Build a Windows automation agent that: receives a JSON task, opens the correct app using UIA, performs the task, takes a before/after screenshot, OCRs the results, and returns a success/failure report.
- **Outcome** Working Windows automation layer – the core engine of your agent

- **Weekly Project** See Day 60 capstone above.

Phase 3 — ■ AI & LLM Engineering

Days 61–120 · Goal: Master LLMs — prompting, tool calling, embeddings, RAG, and fine-tuning. Know how to deploy and call models from OpenAI, Anthropic, Gemini, and local Ollama.

Week 9 | LLM Fundamentals | Days 61–67

Day 61 | How LLMs Work – Transformers

- **Study** Attention Is All You Need (paper); 3Blue1Brown – Neural Networks series
- **Watch** Andrej Karpathy – Let's build GPT (YT)
- **Repo** github.com/karpathy/nanoGPT
- **Exercise** Run nanoGPT locally; train on a small dataset; inspect attention weights
- **Outcome** Intuition for token prediction and attention
- **Paper** Attention Is All You Need (Vaswani et al.)

Day 62 | OpenAI API – Chat, Functions, Streaming

- **Study** OpenAI API Docs – Chat Completions, Function Calling
- **Watch** Dave Ebbelaar – OpenAI API masterclass (YT)
- **Repo** github.com/openai/openai-python
- **Exercise** Build a streaming chat app; use function calling to make the model call a weather API
- **Outcome** Full OpenAI API fluency

Day 63 | Anthropic Claude API + Tool Use

- **Study** Anthropic API Docs – Messages, Tool Use
- **Watch** Prompt Engineering – Claude API guide (YT)
- **Repo** github.com/anthropics/anthropic-sdk-python
- **Exercise** Replicate yesterday's app using Claude; compare tool-calling patterns between APIs
- **Outcome** Multi-provider LLM skills

Day 64 | Local LLMs – Ollama & vLLM

- **Study** Ollama Docs; vLLM Docs – Quickstart
- **Watch** Matt Williams – Ollama guide (YT)
- **Repo** github.com/ollama/ollama
- **Exercise** Run Llama 3 and Mistral locally; benchmark response time vs OpenAI
- **Outcome** Local inference for privacy and cost

Day 65

Prompt Engineering – Advanced Patterns

- **Study** Prompt Engineering Guide (promptingguide.ai); Anthropic Prompt Library
- **Watch** AI Explained – Prompt Engineering (YT)
- **Repo** github.com/dair-ai/Prompt-Engineering-Guide
- **Exercise** Implement: zero-shot, few-shot, chain-of-thought, self-consistency, and ToT prompts
- **Outcome** Extract maximum capability from any model
- **Paper** Chain-of-Thought Prompting (Wei et al.)

Day 66

Structured Outputs & JSON Schema

- **Study** OpenAI Structured Outputs docs; Instructor library docs
- **Watch** Jason Liu – Instructor tutorial (YT)
- **Repo** github.com/jxnl/instructor
- **Exercise** Make LLM always return typed Pydantic objects; build a resume parser
- **Outcome** Reliable structured data from LLMs

Day 67

Embeddings & Semantic Search

- **Study** OpenAI Embeddings docs; text-embedding-3 paper
- **Watch** James Briggs – Embeddings explained (YT)
- **Repo** github.com/openai/openai-python
- **Exercise** Embed 5000 documents; build semantic search returning top-5 results in <100ms
- **Outcome** Vector search over any corpus

■ Weekly Project

Build an AI-powered document Q&A; system: upload PDFs, chunk + embed them in Qdrant, answer questions with citations using RAG.

Week 10 | RAG & Knowledge Systems | Days 68–74

Day 68

RAG Architecture – Naive to Advanced

- **Study** LlamaIndex Docs – RAG concepts; Advanced RAG (Pinecone blog)
- **Watch** Sam Witteveen – RAG tutorial (YT)
- **Repo** github.com/run-llama/llama_index
- **Exercise** Build naive RAG, then add re-ranking with Cohere, then HyDE (hypothetical document embeddings)
- **Outcome** Multi-stage RAG pipeline
- **Paper** Retrieval-Augmented Generation (Lewis et al.)

Day 69	Chunking Strategies
■ Study	LangChain TextSplitters docs; Chroma chunking guide
■ Watch	Greg Kamradt – chunking strategies (YT)
■ Repo	github.com/langchain-ai/langchain
■ Exercise	Compare fixed-size, recursive, semantic, and sentence-window chunking on the same corpus
■ Outcome	Optimal chunking for your use case
Day 70	Hybrid Search – BM25 + Vector
■ Study	Qdrant Hybrid Search docs; Elasticsearch BM25 guide
■ Watch	James Briggs – Hybrid search (YT)
■ Repo	github.com/qdrant/qdrant
■ Exercise	Add BM25 keyword search alongside vector search; merge results with RRF
■ Outcome	Best of both retrieval worlds
Day 71	Document Processing – PDF, Word, Excel
■ Study	pdfplumber docs; python-docx docs; openpyxl docs
■ Watch	Patrick Loeber – Document parsing (YT)
■ Repo	github.com/VikParuchuri/marker
■ Exercise	Build a universal document loader that handles PDF, DOCX, XLSX, HTML, markdown
■ Outcome	Ingest any file format into your RAG pipeline
Day 72	Knowledge Graphs with Neo4j
■ Study	Neo4j Docs – Cypher intro; GraphRAG (Microsoft paper)
■ Watch	Pavan Mirla – Neo4j Python (YT)
■ Repo	github.com/microsoft/graphrag
■ Exercise	Extract entities from documents using an LLM; store in Neo4j; query with Cypher
■ Outcome	Graph-enhanced retrieval for complex queries
■ Paper	From Local to Global: A Graph RAG Approach (Microsoft)
Day 73	Reranking & Query Expansion
■ Study	Cohere Rerank docs; RAG Fusion paper
■ Watch	Whimsical AI – RAG Fusion explained (YT)
■ Repo	github.com/langchain-ai/langchain
■ Exercise	Add Cohere reranker to your pipeline; implement multi-query expansion
■ Outcome	Significantly higher retrieval precision
■ Paper	RAG-Fusion

Day 74

Evaluation – RAGAS & Benchmarking

- **Study** RAGAS docs; TruLens docs
- **Watch** Hamza Farooq – RAG evaluation (YT)
- **Repo** github.com/explodinggradients/ragas
- **Exercise** Evaluate your RAG pipeline with RAGAS: faithfulness, answer relevancy, context recall
- **Outcome** Measurable RAG quality
- **Paper** RAGAS paper

- **Weekly Project** Build a RAG chatbot over all your study notes from Month 1–2. Must score >0.85 faithfulness on RAGAS eval.

Week 11 | LangChain & LangGraph | Days 75–81

Day 75

LangChain – Chains & LCEL

- **Study** LangChain Docs – LCEL; Expression Language guide
- **Watch** Sam Witteveen – LangChain 2024 (YT)
- **Repo** github.com/langchain-ai/langchain
- **Exercise** Build 5 different chains using LCEL: simple, parallel, branching, routing, fallback
- **Outcome** Composable LLM pipelines

Day 76

LangChain – Tools & Agents

- **Study** LangChain Docs – Tools, AgentExecutor
- **Watch** Nicholas Renotte – LangChain agents (YT)
- **Repo** github.com/langchain-ai/langchain
- **Exercise** Build a ReAct agent with 5 custom tools: search, calculator, file read, API call, code exec
- **Outcome** Tool-using agent foundation

Day 77

LangGraph – Stateful Agents

- **Study** LangGraph Docs – Graph, State, Nodes, Edges
- **Watch** LangChain – LangGraph tutorial (official YT)
- **Repo** github.com/langchain-ai/langgraph
- **Exercise** Port yesterday's ReAct agent to LangGraph; add checkpointing and human-in-the-loop
- **Outcome** Stateful, resumable agent graphs

Day 78

LangGraph – Multi-Agent Supervisor

- **Study** LangGraph Docs – Multi-agent, Supervisor pattern
- **Watch** LangChain – Multi-agent with LangGraph (YT)
- **Repo** github.com/langchain-ai/langgraph
- **Exercise** Build a supervisor agent that routes tasks to: research, code, file, and browser sub-agents
- **Outcome** Multi-agent orchestration

Day 79

LiteLLM – Universal LLM Gateway

- **Study** LiteLLM Docs; LiteLLM proxy docs
- **Watch** BerriAI – LiteLLM demo (YT)
- **Repo** github.com/BerriAI/litellm
- **Exercise** Wrap all your LLM calls with LiteLLM; add fallbacks: GPT-4o → Claude → Ollama
- **Outcome** Provider-agnostic LLM layer

Day 80

Model Context Protocol (MCP)

- **Study** Anthropic MCP Docs; MCP spec on GitHub
- **Watch** Fireship – MCP explained (YT)
- **Repo** github.com/modelcontextprotocol/python-sdk
- **Exercise** Build a custom MCP server with 3 tools: file system, web search, Python REPL
- **Outcome** Standard tool protocol for any agent

Day 81

OpenAI Agents SDK

- **Study** OpenAI Agents SDK Docs – Quickstart, Handoffs, Guardrails
- **Watch** OpenAI – Agents SDK demo (YT)
- **Repo** github.com/openai/openai-agents-python
- **Exercise** Build a triage agent that classifies user requests and hands off to specialist agents
- **Outcome** Handoff-based multi-agent systems

- **Weekly Project** Build a research assistant agent using LangGraph that: searches the web, reads papers, stores findings in Qdrant, and generates a structured report.

Week 12 | Agent Architectures & Memory | Days 82–90

Day 82

ReAct Pattern – Implementation

- **Study** ReAct paper; LangChain ReAct agent source
- **Watch** Prompt Engineering – ReAct agent (YT)
- **Repo** github.com/hwchase17/langchain
- **Exercise** Implement ReAct from scratch without any framework; just Python + OpenAI API
- **Outcome** Deep understanding of Thought/Action/Observation loop
- **Paper** ReAct: Synergizing Reasoning and Acting in LLMs

Day 83

Reflexion & Self-Critique

- **Study** Reflexion paper; Self-Refine paper
- **Watch** AI Explained – Reflexion (YT)
- **Repo** github.com/noahshinn/reflexion
- **Exercise** Add a Critic agent that reviews each action before execution and after failure
- **Outcome** Self-improving agent loops
- **Paper** Reflexion: Language Agents with Verbal Reinforcement

Day 84

Plan-and-Execute Architecture

- **Study** Plan-and-Execute paper; BabyAGI architecture
- **Watch** Harrison Chase – Plan and Execute (YT)
- **Repo** github.com/langchain-ai/langgraph
- **Exercise** Build a planner that creates a 10-step task list, then executes each step with re-planning on failure
- **Outcome** Long-horizon task execution
- **Paper** Plan-and-Solve Prompting

Day 85

Memory Systems – Short & Long Term

- **Study** Generative Agents paper; MemGPT paper
- **Watch** AI Jason – Agent memory (YT)
- **Repo** github.com/cpacker/MemGPT
- **Exercise** Implement: (1) conversation buffer, (2) sliding window, (3) summary memory, (4) vector memory
- **Outcome** Four memory patterns fully implemented
- **Paper** Generative Agents: Interactive Simulacra of Human Behavior

Day 86**Episodic Memory & Experience Replay**

■ Study	Voyager paper; GITM paper
■ Watch	Two Minute Papers – Voyager (YT)
■ Repo	github.com/MineDreameer/MineDreameer
■ Exercise	Give your agent an experience store: save (task, plan, result, reflection) and retrieve similar past experiences before planning
■ Outcome	Agent that improves from experience
■ Paper	Voyager: An Open-Ended Embodied Agent with LLMs

Day 87**Tool Design Best Practices**

■ Study	Toolformer paper; Gorilla paper
■ Watch	James Briggs – Tool calling best practices (YT)
■ Repo	github.com/ShishirPatil/gorilla
■ Exercise	Design a tool registry with: name, description, JSON schema, examples, error messages. Build 10 tools for your Windows agent.
■ Outcome	Well-designed tool ecosystem
■ Paper	Toolformer: LLMs Can Teach Themselves to Use Tools

Day 88**PydanticAI – Type-Safe Agents**

■ Study	PydanticAI Docs – Agents, Tools, Results
■ Watch	Samuel Colvin – PydanticAI intro (YT)
■ Repo	github.com/pydantic/pydantic-ai
■ Exercise	Port your ReAct agent to PydanticAI; use typed results and dependency injection
■ Outcome	Production-quality typed agent code

Day 89**SmolAgents & Minimal Frameworks**

■ Study	SmolAgents Docs; HuggingFace Transformers Agents
■ Watch	HuggingFace – SmolAgents (YT)
■ Repo	github.com/huggingface/smolagents
■ Exercise	Build a CodeAct agent with SmolAgents that writes + executes Python to complete tasks
■ Outcome	Code-as-action agent pattern
■ Paper	CodeAct: Code-based Agent Actions

Day 90

Month 3 Capstone – Full Agent Stack

■ Study	Review all Month 3 topics
■ Watch	—
■ Repo	—
■ Exercise	Build a complete AI assistant: LangGraph graph → multiple tools (web, file, code, Windows) → Qdrant memory → Neo4j knowledge graph → LiteLLM backend → FastAPI server → React UI
■ Outcome	Production-ready agent foundation

■ **Weekly Project** See Day 90 capstone.

Phase 4 — ■ Browser Automation & Computer Vision

Days 121–180 · Goal: Build an agent that can navigate any website, fill forms, download files, solve CAPTCHAs, and extract data — and one that can 'see' the screen like a human does.

Week 13 | Playwright & Browser Control | Days 121–127

Day 121 | Playwright – Basics & Browser Contexts

- **Study** Playwright Python Docs – Getting Started, Selectors
- **Watch** Playwright – Official tutorial (YT)
- **Repo** github.com/microsoft/playwright-python
- ■ **Exercise** Automate Google search → click first result → extract page title + text
- **Outcome** Core Playwright automation

Day 122 | Playwright – Forms, Auth & Cookies

- **Study** Playwright Docs – Authentication, Storage State
- **Watch** TestingBot – Playwright authentication (YT)
- **Repo** github.com/microsoft/playwright-python
- ■ **Exercise** Log in to GitHub, save auth state, reuse in next session without re-logging
- **Outcome** Persistent authenticated sessions

Day 123 | CDP – Chrome DevTools Protocol

- **Study** CDP Docs; Playwright CDP docs
- **Watch** Umar Hansa – DevTools Protocol (YT)
- **Repo** github.com/pyppeteer/pyppeteer
- ■ **Exercise** Use CDP to: intercept network requests, inject JavaScript, emulate devices
- **Outcome** Deep browser control via CDP

Day 124 | Playwright – File Downloads & Uploads

- **Study** Playwright Docs – Downloads, File Chooser
- **Watch** Automation Bro – Playwright downloads (YT)
- **Repo** github.com/microsoft/playwright-python
- ■ **Exercise** Download a PDF from a site; upload a file to a form; verify success
- **Outcome** File transfer automation

Day 125 | Playwright – Waiting, Retries & Reliability

- **Study** Playwright Docs – Auto-waiting, Timeouts, Retries
- **Watch** Checkly – Reliable Playwright (YT)
- **Repo** github.com/microsoft/playwright-python
- **Exercise** Build a resilient scraper that handles network errors, element not found, and timeouts gracefully
- **Outcome** Production-reliable browser automation

Day 126 | Browser Use – AI-Driven Browser Agent

- **Study** Browser Use Docs; Browser Use paper
- **Watch** browser-use – demo video (YT)
- **Repo** github.com/browser-use/browser-use
- **Exercise** Use Browser Use to complete 10 real tasks: search, login, extract, fill form, navigate multi-page flow
- **Outcome** AI-native browser agent

Day 127 | Skyvern – Vision-Based Web Agent

- **Study** Skyvern Docs; Skyvern paper
- **Watch** Skyvern – demo (YT)
- **Repo** github.com/Skyvern-AI/skyvern
- **Exercise** Run Skyvern locally; give it 5 natural language browser tasks and evaluate success rate
- **Outcome** Vision-guided web navigation

- **Weekly Project** Build a browser agent that monitors a job board daily, extracts matching listings, fills an auto-apply form, and emails a summary.

Week 14 | Data Extraction & Web Intelligence | Days 128–134

Day 128 | HTML Parsing – BeautifulSoup & Parsel

- **Study** bs4 docs; parsel docs (Scrapy selector engine)
- **Watch** John Watson Rooney – Web scraping (YT)
- **Repo** github.com/scrapy/parsel
- **Exercise** Scrape a news site: extract article title, author, date, body for 100 articles
- **Outcome** Robust HTML parsing

Day 129 | Scrapy – Industrial Scraping

- **Study** Scrapy Docs – Spiders, Pipelines, Middlewares
- **Watch** John Watson Rooney – Scrapy tutorial (YT)
- **Repo** github.com/scrapy/scrapy
- **Exercise** Build a Scrapy spider with: item pipelines, Postgres storage, rotating proxies
- **Outcome** Production-scale scraping

Day 130 | Anti-Bot Bypass Techniques

- **Study** Playwright-Stealth docs; cloudscraper docs
- **Watch** Leon – anti-bot techniques (YT)
- **Repo** github.com/ultrafunkamsterdam/undetected-chromedriver
- **Exercise** Bypass Cloudflare, solve hCaptcha (via 2captcha API), fingerprint evasion
- **Outcome** Navigate heavily protected sites

Day 131 | PDF & Document Extraction

- **Study** pdfplumber docs; Marker docs
- **Watch** David Shapiro – document extraction (YT)
- **Repo** github.com/VikParuchuri/marker
- **Exercise** Build a pipeline: download PDF, extract tables + text, store in structured JSON
- **Outcome** Any document type into structured data

Day 132 | Screenshot + Vision for Web Pages

- **Study** Playwright screenshot docs; GPT-4V vision API
- **Watch** Matthew Berman – GPT-4 Vision (YT)
- **Repo** github.com/openai/openai-python
- **Exercise** Screenshot a web page, send to GPT-4V, ask it to identify form fields and their labels
- **Outcome** Vision-guided form understanding

Day 133 | Search Engine APIs & SerpAPI

- **Study** SerpAPI Docs; Brave Search API docs
- **Watch** Patrick Loeber – SerpAPI (YT)
- **Repo** github.com/serpapi/google-search-results-python
- **Exercise** Build a research tool: query Brave + Google, deduplicate results, rank by relevance
- **Outcome** Multi-source web intelligence

Day 134 | Graph-Based Web Crawling

- **Study** Study web graph crawl algorithms; BFS vs DFS crawling
- **Watch** Hussein Nasser – Web crawler architecture (YT)
- **Repo** github.com/ColinEberhardt/wasmboy
- **Exercise** Build a depth-limited crawler that maps all internal links on a site as a graph in Neo4j
- **Outcome** Site topology understanding

- **Weekly Project** Build a competitive intelligence bot: crawl 10 competitor sites, extract pricing/features, store in Postgres, generate a weekly comparison report.

Week 15 | Computer Vision – Deep Dive | Days 135–141

Day 135 | OpenCV – Image Processing Fundamentals

- **Study** OpenCV Python Docs – Core Operations, Image Transforms
- **Watch** sentdex – OpenCV full course (YT)
- **Repo** github.com/opencv/opencv-python
- **Exercise** Implement: edge detection, contour finding, colour filtering, perspective transform
- **Outcome** Core OpenCV image manipulation

Day 136 | YOLO – Object Detection

- **Study** YOLOv8 Docs; Ultralytics YOLO tutorial
- **Watch** Nicholas Renotte – YOLO (YT)
- **Repo** github.com/ultralytics/ultralytics
- **Exercise** Train YOLO on a custom dataset of UI elements (buttons, inputs, checkboxes)
- **Outcome** Custom UI element detection

Day 137 | SAM2 – Segment Anything

- **Study** SAM2 paper; Meta SAM2 docs
- **Watch** Roboflow – SAM2 tutorial (YT)
- **Repo** github.com/facebookresearch/segment-anything-2
- **Exercise** Use SAM2 to segment every UI element on a screenshot without any training
- **Outcome** Zero-shot UI segmentation
- **Paper** Segment Anything 2 (Meta)

Day 138 | Florence-2 – Vision Understanding

■ Study	Florence-2 paper; HuggingFace model card
■ Watch	Mervin Praisson – Florence-2 (YT)
■ Repo	github.com/microsoft/unilm
■ Exercise	Use Florence-2 to: caption a screenshot, detect UI elements, answer questions about what's on screen
■ Outcome	All-in-one vision understanding
■ Paper	Florence-2 (Microsoft)

Day 139 | Grounding DINO – Open-Vocabulary Detection

■ Study	Grounding DINO paper; HuggingFace docs
■ Watch	Roboflow – Grounding DINO (YT)
■ Repo	github.com/IDEA-Research/GroundingDINO
■ Exercise	Detect UI elements using text prompts: find('the submit button'), find('the password field')
■ Outcome	Natural language-guided element detection
■ Paper	Grounding DINO

Day 140 | OmniParser – Screen Parsing Pipeline

■ Study	OmniParser paper; Microsoft OmniParser repo
■ Watch	Microsoft Research – OmniParser (YT)
■ Repo	github.com/microsoft/OmniParser
■ Exercise	Run OmniParser on 20 different screenshots; evaluate quality; integrate into your agent
■ Outcome	Production screen understanding
■ Paper	OmniParser for Pure Vision Based GUI Agent

Day 141 | Screen Recording & Replay

■ Study	mss docs (fast screenshots); pyscreenrec docs
■ Watch	Tech With Tim – Screen recording Python (YT)
■ Repo	github.com/BoboTiG/python-mss
■ Exercise	Build a screen recorder that captures at 10fps, detects UI changes, and generates a change log
■ Outcome	Continuous screen monitoring pipeline

- | | |
|------------------|---|
| ■ Weekly Project | Build a universal GUI understanding API: POST a screenshot, receive back structured JSON of all detected elements with type, label, bounding box, and confidence. |
|------------------|---|

Week 16 | Integration – Browser + Vision + LLM | Days 142–150

Day 142 | Browser Agent Architecture

■ Study	WebArena paper; WebVoyager paper
■ Watch	WorldModel – WebArena overview (YT)
■ Repo	github.com/web-arena-x/webarena
■ Exercise	Design and document your browser agent architecture: perception → planning → action → memory
■ Outcome	Complete agent design document
■ Paper	WebArena: A Realistic Web Environment for Agents

Day 143 | Screen Grounding – Click by Description

■ Study	SeeClick paper; ScreenSpot benchmark
■ Watch	HuggingFace – SeeClick (YT)
■ Repo	github.com/njucckevin/SeeClick
■ Exercise	Build: given 'click the blue submit button', vision model locates it, returns normalized coordinates, agent clicks
■ Outcome	Natural language → screen coordinate
■ Paper	SeeClick: Harnessing GUI Grounding for Advanced AI

Day 144 | Multi-Step Web Task Execution

■ Study	Mind2Web paper; OSWorld paper
■ Watch	CMU – OSWorld benchmark (YT)
■ Repo	github.com/OSWorld-Org/OSWorld
■ Exercise	Execute 5 multi-step tasks: book a flight, fill a complex form, extract and transform data
■ Outcome	Multi-step task completion at >80% success
■ Paper	OSWorld: Benchmarking Multimodal AI Agents

Day 145 | Self-Healing Selectors

■ Study	Playwright locator strategies; Self-healing test concepts
■ Watch	Applitools – Self-healing selectors (YT)
■ Repo	github.com/microsoft/playwright
■ Exercise	Build selector fallback chain: ID → name → aria-label → CSS → text → vision-based
■ Outcome	Resilient element location that survives UI changes

Day 146	Browser Extension Integration
■ Study	Chrome Extension Docs – Manifest V3; CDP extension support
■ Watch	Jack Harrington – Chrome Extension (YT)
■ Repo	github.com/nicehash/NiceHashQuickMiner
■ Exercise	Build a Chrome extension that exposes a WebSocket API for your Python agent to control
■ Outcome	Deep browser integration via extension
Day 147	Parallel Browser Agents
■ Study	Playwright browser context isolation; asyncio multi-page
■ Watch	Microsoft – Playwright parallelism (YT)
■ Repo	github.com/microsoft/playwright-python
■ Exercise	Run 5 browser agents in parallel on different tasks; aggregate results
■ Outcome	Concurrent browser automation
Day 148	CAPTCHA Handling Strategies
■ Study	2captcha API docs; CapSolver docs; Anti-Captcha docs
■ Watch	Caleb Curry – CAPTCHA solving (YT)
■ Repo	github.com/2captcha/2captcha-python
■ Exercise	Handle: reCAPTCHA v2, v3, hCaptcha, image CAPTCHA via APIs + human fallback
■ Outcome	CAPTCHA pipeline with human escalation
Day 149	Proxy & Session Management
■ Study	Playwright proxy docs; Rotating proxy guide
■ Watch	John Watson Rooney – Proxy rotation (YT)
■ Repo	github.com/mahdibland/V2RayAggregator
■ Exercise	Integrate rotating proxies; implement session pools; detect and recover from bans
■ Outcome	Scale scraping without detection
Day 150	Month 5 Capstone – Browser Agent
■ Study	Review all browser + vision content
■ Watch	—
■ Repo	—
■ Exercise	Build a complete browser agent: given a natural language task, it uses OmniParser to see the screen, plans actions with GPT-4, executes with Playwright, retries failures with vision re-grounding, and stores results in Qdrant.
■ Outcome	Production browser agent that can navigate any website

Phase 5 — ■ Multi-Agent Systems & Production

Days 181–270 · Goal: Build the full multi-agent Windows OS agent: planner, executor, critic, memory, browser, code, file, and terminal agents — all orchestrated together in production.

Week 17 | Multi-Agent Orchestration | Days 181–187

Day 181 | AutoGen Framework

- **Study** AutoGen Docs; AutoGen paper
- **Watch** Microsoft – AutoGen tutorial (YT)
- **Repo** github.com/microsoft/autogen
- **Exercise** Build a 3-agent system: UserProxy, AssistantAgent, CriticAgent for code generation + review
- **Outcome** AutoGen conversation patterns
- **Paper** AutoGen: Enabling Next-Gen LLM Applications

Day 182 | CrewAI – Role-Based Agents

- **Study** CrewAI Docs; CrewAI examples
- **Watch** AI Jason – CrewAI tutorial (YT)
- **Repo** github.com/crewAIInc/crewAI
- **Exercise** Build a content creation crew: Researcher + Writer + Editor + Publisher agents
- **Outcome** Hierarchical crew with role delegation

Day 183 | MetaGPT – Software Team Simulation

- **Study** MetaGPT paper; MetaGPT Docs
- **Watch** Dave Ebbelaar – MetaGPT (YT)
- **Repo** github.com/geekan/MetaGPT
- **Exercise** Use MetaGPT to build a simple app: watch how it creates PRD, architecture, code, tests
- **Outcome** Software team agent pattern
- **Paper** MetaGPT: Meta Programming for Multi-Agent Collaboration

Day 184 | Designing Your Agent Architecture

- **Study** OpenHands paper; SWE-Agent paper
- **Watch** Robert Luciani – Agent architecture (YT)
- **Repo** github.com/All-Hands-AI/OpenHands
- **Exercise** Draw and document your full agent architecture: 20 agents, their roles, communication protocols, shared memory
- **Outcome** Your personal agent blueprint
- **Paper** SWE-agent: Agent Computer Interfaces Enable Automated Software Engineering

Day 185 | Inter-Agent Communication

- **Study** CAMEL paper; A2A protocol docs
- **Watch** Two Minute Papers – CAMEL agents (YT)
- **Repo** github.com/camel-ai/camel
- **Exercise** Implement message passing between agents using Kafka topics; each agent has an inbox and outbox
- **Outcome** Decoupled agent communication
- **Paper** CAMEL: Communicative Agents for Mind Exploration

Day 186 | Task Decomposition & DAG Planning

- **Study** HuggingGPT paper; TaskMatrix paper
- **Watch** Yannic Kilcher – HuggingGPT (YT)
- **Repo** github.com/microsoft/TaskMatrix
- **Exercise** Build a planner that converts any user goal into a DAG of tasks with dependencies
- **Outcome** Automatic task graph generation
- **Paper** HuggingGPT: Solving AI Tasks with ChatGPT

Day 187 | Workflow Engine – LangGraph at Scale

- **Study** LangGraph Docs – Streaming, Async, Checkpointing
- **Watch** LangChain – LangGraph advanced (YT)
- **Repo** github.com/langchain-ai/langgraph
- **Exercise** Build a persistent workflow engine: tasks survive restarts, can be paused/resumed, have timeouts
- **Outcome** Reliable long-running workflows

- **Weekly Project** Build a 5-agent research team that automatically researches any topic, writes a report, fact-checks it, and publishes to a blog.

Week 18 | Windows Agent + Terminal Agent + Code Agent | Days 188–210

Day 188 | Windows Agent – Core Tools

■ Study	Review pywinauto, pywin32, WMI, Win32
■ Watch	—
■ Repo	github.com/pywinauto/pywinauto
■ Exercise	Build complete Windows tool set: open_app, close_app, focus_window, type_text, click_element, capture_screen, read_screen
■ Outcome	20+ Windows action tools

Day 189 | Windows Agent – App-Specific Handlers

■ Study	Win32 API docs per-application; COM automation per-app
■ Watch	—
■ Repo	github.com/microsoft/Windows-classic-samples
■ Exercise	Build dedicated handlers for: Notepad, Explorer, Control Panel, Settings, Calculator
■ Outcome	Reliable control of core Windows apps

Day 190 | Terminal Agent – Command Execution

■ Study	subprocess docs; asyncio.create_subprocess_exec docs
■ Watch	—
■ Repo	github.com/microsoft/terminal
■ Exercise	Build terminal agent: execute any shell command, stream output, handle stdin, detect errors, timeout
■ Outcome	Safe, streaming terminal execution

Day 191 | Terminal Agent – Package Managers

■ Study	winget docs; pip, npm, cargo, choco docs
■ Watch	—
■ Repo	github.com/microsoft/winget-cli
■ Exercise	Agent can install software via winget, pip, npm; verify installation; add to PATH
■ Outcome	Self-modifying agent environment

Day 192 Code Agent – File Reading

- **Study** Tree-sitter docs; pygments docs; AST module docs
- **Watch** Cursor – Agent walkthrough (YT)
- **Repo** github.com/tree-sitter/py-tree-sitter
- **Exercise** Parse Python, JS, Rust, and C++ files; extract function signatures, classes, imports via AST
- **Outcome** Semantic code understanding

Day 193 Code Agent – Editing & Refactoring

- **Study** LibCST docs; Rope docs
- **Watch** ArjanCodes – Code refactoring with AI (YT)
- **Repo** github.com/Instagram/LibCST
- **Exercise** Agent edits code: add a new function, rename a variable across all usages, extract a method
- **Outcome** Safe code editing without string replacement

Day 194 Code Agent – Test Generation

- **Study** pytest docs; Hypothesis docs
- **Watch** Thoughtworks – AI test generation (YT)
- **Repo** github.com/pytest-dev/pytest
- **Exercise** Agent reads any Python function, generates pytest test cases with edge cases, runs them, fixes failures
- **Outcome** Automated test generation loop

Day 195 Code Agent – Git Integration

- **Study** GitPython docs; git CLI reference
- **Watch** Fireship – Git internals (YT)
- **Repo** github.com/gitpython-developers/GitPython
- **Exercise** Agent can: clone repo, create branch, make changes, commit, push, create PR via GitHub API
- **Outcome** Full Git workflow automation

Day 196 VSCode Integration

- **Study** VSCode Extension API; Language Server Protocol docs
- **Watch** Fireship – VSCode extension dev (YT)
- **Repo** github.com/microsoft/vscode-extension-samples
- **Exercise** Build a VSCode extension that exposes your agent's capabilities inside the editor
- **Outcome** IDE-embedded agent

Day 197	Excel Agent – openpyxl & xlwings
■ Study	openpyxl docs; xlwings docs
■ Watch	Calmcode – openpyxl (YT)
■ Repo	github.com/xlwings/xlwings
■ Exercise	Agent opens Excel, reads data, writes formulas, creates charts, formats cells
■ Outcome	Full Excel automation
Day 198	Word & PowerPoint Agent
■ Study	python-docx docs; pptx docs
■ Watch	Learn Automation – python-docx (YT)
■ Repo	github.com/python-openxml/python-docx
■ Exercise	Agent generates a Word report and PowerPoint deck from structured data
■ Outcome	Office document generation
Day 199	Email Agent – Outlook & SMTP
■ Study	win32com Outlook docs; smtplib docs; imaplib docs
■ Watch	Tech With Tim – Email automation (YT)
■ Repo	github.com/mhammond/pywin32
■ Exercise	Agent: read inbox, classify emails, draft replies, send with attachments, create calendar event
■ Outcome	Full email workflow automation
Day 200	PDF Agent
■ Study	pdfplumber docs; pypdf docs; reportlab docs
■ Watch	—
■ Repo	github.com/jsvine/pdfplumber
■ Exercise	Agent reads invoice PDFs, extracts line items, totals, vendor info; generates summary Excel
■ Outcome	Structured PDF data extraction
Day 201	File Agent – Search & Organize
■ Study	pathlib docs; watchdog docs; Everything SDK
■ Watch	—
■ Repo	github.com/gorakhargosh/watchdog
■ Exercise	Agent: find all PDFs modified in last 7 days, move to archive folder, rename by date
■ Outcome	Intelligent file management

Day 202	Calendar Agent
■ Study	win32com Calendar docs; Google Calendar API; icalendar Python
■ Watch	—
■ Repo	github.com/collective/icalendar
■ Exercise	Agent reads your calendar, finds free slots, schedules meetings, sends invites
■ Outcome	Calendar workflow automation
Day 203	Integrating All Agents
■ Study	LangGraph multi-agent docs; OpenAI Agents handoffs
■ Watch	LangChain – Full agent system (YT)
■ Repo	github.com/langchain-ai/langgraph
■ Exercise	Route user commands to correct agent: 'email X' → Email Agent, 'open Y in Excel' → Excel Agent
■ Outcome	Unified agent dispatcher
Day 204	Permission & Safety System
■ Study	Design dangerous action categories
■ Watch	—
■ Repo	—
■ Exercise	Build permission engine: tag each tool as SAFE/MEDIUM/DANGEROUS/CRITICAL; block critical without explicit user confirmation
■ Outcome	Safe agent that never does destructive things without approval
Day 205	Recovery & Retry System
■ Study	Tenacity docs; retry patterns
■ Watch	—
■ Repo	github.com/jd/tenacity
■ Exercise	Add retry logic to every agent action: exponential backoff, screenshot-on-failure, re-plan on 3 consecutive failures
■ Outcome	Self-healing agent execution

Day 206	Human-in-the-Loop
■ Study	LangGraph Docs – Human-in-the-loop; interrupt()
■ Watch	LangChain – HITL (YT)
■ Repo	github.com/langchain-ai/langgraph
■ Exercise	Agent pauses on critical actions, notifies user, waits for approval, then resumes or aborts
■ Outcome	Safe agent with human oversight
Day 207	Voice Interface – STT & TTS
■ Study	OpenAI Whisper docs; OpenAI TTS docs; Faster-Whisper
■ Watch	AssemblyAI – Whisper tutorial (YT)
■ Repo	github.com/openai/whisper
■ Exercise	Add voice: speak a command → Whisper transcribes → agent executes → agent speaks result via TTS
■ Outcome	Voice-controlled Windows agent
Day 208	Workflow Recording & Replay
■ Study	Study macro recording concepts; pyautogui record
■ Watch	—
■ Repo	—
■ Exercise	Record a multi-step workflow (open Excel, fill data, save), extract actions as JSON, replay reliably
■ Outcome	Learn-by-demonstration capability
Day 209	Security – Prompt Injection Defense
■ Study	OWASP LLM Top 10; Prompt injection papers
■ Watch	Simon Willison – Prompt injection (YT)
■ Repo	github.com/greshake/llm-security
■ Exercise	Implement prompt injection defense: input sanitization, system prompt isolation, output validation
■ Outcome	Secure agent architecture
■ Paper	Prompt Injection Attacks and Defenses in LLM-Integrated Applications

Day 210**Month 7 Capstone – Full Windows Agent**

■ Study	Review all agent modules
■ Watch	—
■ Repo	—
■ Exercise	Demo task: 'Find all PDFs from last month, extract expense data, update Excel budget, email summary to my manager, archive PDFs.' Agent should complete this fully autonomously in <5 minutes.
■ Outcome	End-to-end Windows agent working on real tasks

■ **Weekly Project** See Day 210 capstone.

Phase 6 — ■ Production, Polish & Capstone

Days 271–365 · Goal: Turn your agent into a production-grade system: observability, Docker, CI/CD, auto-updates, plugin system, cloud sync, and a polished UI. Build your final Jarvis OS capstone.

Week 19 | Production Engineering | Days 271–300

Day 271 | Structured Logging & Tracing

- **Study** OpenTelemetry Python docs; structlog docs
- **Watch** TechWorld with Nana – Observability (YT)
- **Repo** github.com/open-telemetry/opentelemetry-python
- **Exercise** Add OTel traces to every agent action; view flame graphs in Jaeger
- **Outcome** Full request tracing across all agents

Day 272 | Metrics & Alerting – Prometheus + Grafana

- **Study** Prometheus Python client docs; Grafana dashboard guide
- **Watch** TechWorld with Nana – Prometheus (YT)
- **Repo** github.com/prometheus/client_python
- **Exercise** Track: task success rate, average completion time, tool error rate, LLM latency
- **Outcome** Real-time agent performance dashboard

Day 273 | Docker Compose – Full Stack

- **Study** Docker Compose v3 docs; multi-service networking
- **Watch** NetworkChuck – Docker Compose (YT)
- **Repo** github.com/docker/compose
- **Exercise** Containerise everything: FastAPI, LangGraph, Qdrant, Neo4j, Redis, Postgres, Grafana in one docker-compose.yml
- **Outcome** One-command deployment of full stack

Day 274 | CI/CD – GitHub Actions

- **Study** GitHub Actions Docs; act (local GHA runner)
- **Watch** TechWorld with Nana – GitHub Actions (YT)
- **Repo** github.com/nektos/act
- **Exercise** Build CI pipeline: lint → test → build Docker image → push to registry → deploy
- **Outcome** Automated deployment on every commit

Day 275 Plugin Architecture

- **Study** Pluggy docs; Python plugin patterns
- **Watch** ArjanCodes – Plugin system (YT)
- **Repo** github.com/pytest-dev/pluggy
- **Exercise** Build plugin system: third-party developers can add new agents/tools as Python packages
- **Outcome** Extensible, marketplace-ready architecture

Day 276 Auto-Updates – Tauri Updater

- **Study** Tauri v2 Updater docs; Sparkle framework concepts
- **Watch** Tauri – Updater tutorial (YT)
- **Repo** github.com/tauri-apps/tauri
- **Exercise** Implement auto-update: app checks GitHub releases, downloads delta update, applies without restart
- **Outcome** Seamless over-the-air updates

Day 277 Cloud Sync – Settings & Memory

- **Study** Supabase Docs; Cloudflare R2 docs
- **Watch** Supabase – Python integration (YT)
- **Repo** github.com/supabase/supabase-py
- **Exercise** Sync agent memory and settings to Supabase; restore on new machine in <30 seconds
- **Outcome** Cross-device agent continuity

Day 278 Local LLM Integration – Privacy Mode

- **Study** Ollama Docs; llama.cpp docs; mlx-lm docs
- **Watch** Matt Williams – Local LLMs (YT)
- **Repo** github.com/abetlen/llama-cpp-python
- **Exercise** Add privacy mode: all data stays local using Ollama + llama.cpp; zero data leaves device
- **Outcome** Enterprise-safe local deployment option

Day 279 Multi-Machine Remote Execution

- **Study** Paramiko docs; SSH tunneling; Celery distributed tasks
- **Watch** Hussein Nasser – SSH tunneling (YT)
- **Repo** github.com/celery/celery
- **Exercise** Agent can execute tasks on remote machines via SSH; results streamed back in real-time
- **Outcome** Distributed agent fleet

Day 280 Rate Limiting & Cost Management

- **Study** LiteLLM budget tracking; token counting with tiktoken
- **Watch** —
- **Repo** github.com/openai/tiktoken
- **Exercise** Add budget controls: daily spend cap, token counting per task, automatic fallback to cheaper model
- **Outcome** Cost-controlled production agent

Day 281 Task Scheduling & Cron Jobs

- **Study** APScheduler docs; Temporal.io intro
- **Watch** Coding Tech – APScheduler (YT)
- **Repo** github.com/agronholm/apscheduler
- **Exercise** Add scheduled tasks: daily report at 9am, weekly backup on Sunday, monthly summary
- **Outcome** Autonomous scheduled execution

Day 282 Notification System

- **Study** Windows Toast notification docs; Telegram Bot API; SendGrid
- **Watch** Tech With Tim – Windows notifications Python (YT)
- **Repo** github.com/jithurjacob/Windows-10-Toast-Notifications
- **Exercise** Agent notifies via: Windows toast, Telegram message, email — on task completion and error
- **Outcome** Multi-channel notification pipeline

Day 283 API Gateway & Security

- **Study** FastAPI security docs; JWT docs; API key management
- **Watch** Patrick Loeber – FastAPI auth (YT)
- **Repo** github.com/tiangolo/fastapi
- **Exercise** Add: JWT auth, API key management, rate limiting, CORS, input validation to your API
- **Outcome** Production-secure API

Day 284 Performance Optimisation

- **Study** Python profiling docs; line_profiler; memory_profiler
- **Watch** mCoding – Python performance (YT)
- **Repo** github.com/joerick/pyinstrument
- **Exercise** Profile your agent, identify top 3 bottlenecks, optimise to 2x speedup
- **Outcome** Performance-tuned production system

Day 285	Tauri UI – Polished Desktop App
■ Study	Tauri v2 Docs; shadcn/ui components; Tailwind CSS
■ Watch	Kevin Powell – Tailwind CSS (YT)
■ Repo	github.com/shadcn-ui/ui
■ Exercise	Build production UI: chat interface, task timeline, agent status dashboard, settings panel, activity log
■ Outcome	Professional desktop application
Day 286	Accessibility & Internationalisation
■ Study	WAI-ARIA docs; i18next docs
■ Watch	Scrimba – A11y (YT)
■ Repo	github.com/i18next/i18next
■ Exercise	Add keyboard navigation, screen reader support, and English/Hindi localisation
■ Outcome	Inclusive, globally-ready application
Day 287	Testing – E2E with Playwright
■ Study	Playwright Test docs; Component testing docs
■ Watch	Fireship – Playwright testing (YT)
■ Repo	github.com/microsoft/playwright
■ Exercise	Write E2E tests for 20 agent workflows; run in CI pipeline
■ Outcome	Regression-proof agent system
Day 288	Documentation & Onboarding
■ Study	MkDocs Material docs; Mintlify docs
■ Watch	—
■ Repo	github.com/squidfunk/mkdocs-material
■ Exercise	Write: architecture doc, API reference, user guide, video walkthrough, README
■ Outcome	Professional open-source project
Day 289	Load Testing & Stress Testing
■ Study	Locust docs; k6 docs
■ Watch	TechWorld with Nana – k6 load testing (YT)
■ Repo	github.com/locustio/locust
■ Exercise	Load test your API: 100 concurrent users, measure p99 latency, identify breaking point
■ Outcome	Known performance limits

Day 290

Security Audit

- **Study** OWASP Top 10; Bandit Python security linter docs
- **Watch** Fireship – Security for developers (YT)
- **Repo** github.com/PyCQA/bandit
- **Exercise** Run Bandit + Trivy + OWASP ZAP on your codebase; fix all HIGH/CRITICAL findings
- **Outcome** Security-hardened application

Day 291

Open Source Preparation

- **Study** GitHub – How to make a great README; Choose a license
- **Watch** Eddie Jaoude – Open Source (YT)
- **Repo** github.com/EddieJaoudeCommunity
- **Exercise** Prepare open-source release: MIT license, CONTRIBUTING.md, issue templates, GitHub Actions CI
- **Outcome** Ready for public release

Day 292

Demo Recording & Portfolio

- **Study** OBS Studio docs; Loom recording guide
- **Watch** —
- **Repo** —
- **Exercise** Record a 5-minute product demo showing 10 impressive agent capabilities. Publish to YouTube.
- **Outcome** Portfolio-quality demo video

- **Weekly Project** Production readiness sprint — achieve all of the above.

Week 20 | Final Capstone – Jarvis OS | Days 293–365

Day 293

Capstone Kickoff – Architecture Review

- **Study** Review all 292 days of work
- **Watch** —
- **Repo** —
- **Exercise** Document final architecture; list all 20 agents; draw interaction diagram; set success criteria
- **Outcome** Capstone project fully scoped

Day 300	Core Loop Integration
■ Study	LangGraph streaming docs
■ Watch	—
■ Repo	—
■ Exercise	Integrate all agents into single LangGraph: user speaks → planner → agents execute → critic reviews → memory stores → user informed
■ Outcome	Complete agent loop working end-to-end
Day 310	Benchmark Against OSWorld
■ Study	OSWorld benchmark; WindowsAgentArena benchmark
■ Watch	—
■ Repo	github.com/OSWorld-Org/OSWorld
■ Exercise	Run 50 OSWorld tasks; measure success rate; compare to published baselines
■ Outcome	Quantified agent performance
■ Paper	OSWorld
Day 320	Plugin Marketplace MVP
■ Study	Plugin ecosystem design patterns
■ Watch	—
■ Repo	—
■ Exercise	Ship 5 community plugins: Calendar, Spotify, Gmail, GitHub, Notion; publish API docs
■ Outcome	Extensible plugin ecosystem
Day 330	Mobile Companion App
■ Study	React Native docs; Expo docs
■ Watch	Traversy Media – React Native (YT)
■ Repo	github.com/expo/expo
■ Exercise	Build iOS/Android companion app: send tasks to desktop agent, receive updates, approve actions
■ Outcome	Cross-device agent control

Day 340 Self-Improvement Loop

■ Study	Self-play RL concepts; RLHF overview
■ Watch	Andrej Karpathy – RLHF (YT)
■ Repo	github.com/openai/lm-human-preferences
■ Exercise	Agent logs failed tasks, identifies patterns, asks LLM to generate better strategies, tests them
■ Outcome	Agent that continuously improves

Day 350 Beta Release

■ Study	Product hunt launch guide; GitHub releases
■ Watch	—
■ Repo	—
■ Exercise	Launch on Product Hunt, GitHub, HackerNews; collect 50 beta users; gather feedback
■ Outcome	Real-world user feedback loop

Day 360 Iteration Based on Feedback

■ Study	User research; prioritisation frameworks
■ Watch	—
■ Repo	—
■ Exercise	Fix top 10 issues from beta; improve task success rate by 10%
■ Outcome	Product quality substantially improved

Day 365 Day 365 – Demo Day

■ Study	—
■ Watch	—
■ Repo	—
■ Exercise	Record final demo: give your agent 10 complex tasks covering every domain. Share online. Write a blog post about your 365-day journey.
■ Outcome	A production-quality Windows AI Agent that can genuinely be compared to OpenHands, Manus, and Cursor Agent

■ **Weekly Project** Jarvis OS – your completed Windows AI Agent.

Resource Library

■ Python

- Fluent Python – Luciano Ramalho
- Python Cookbook – Beazley & Jones
- Effective Python – Brett Slatkin
- High Performance Python – Gorelick & Ozsvald

■ Systems

- Windows Internals Part 1 & 2 – Russinovich
- Programming Windows – Petzold
- The Rust Programming Language – official
- Operating Systems: Three Easy Pieces – Arpaci-Dusseau
- Modern Operating Systems – Tanenbaum

■ Data & Architecture

- Designing Data-Intensive Applications – Kleppmann
- Clean Architecture – Robert C. Martin
- Database Internals – Petrov
- Patterns of Enterprise Application Architecture – Fowler

■ AI & ML

- Artificial Intelligence: A Modern Approach – Russell & Norvig
- Deep Learning – Goodfellow, Bengio & Courville
- Hands-On Machine Learning – Géron
- Natural Language Processing with Transformers – Tunstall et al.

■ Networking

- Computer Networking: A Top-Down Approach – Kurose & Ross
- HTTP: The Definitive Guide – Gourley et al.

Key Research Papers (in Reading Order)

- Attention Is All You Need – Vaswani et al. (2017)
- ReAct: Synergizing Reasoning and Acting – Yao et al. (2022)
- Reflexion: Language Agents with Verbal Reinforcement – Shinn et al. (2023)
- Chain-of-Thought Prompting – Wei et al. (2022)
- Toolformer: LLMs Can Teach Themselves to Use Tools – Schick et al. (2023)
- Retrieval-Augmented Generation – Lewis et al. (2020)
- Generative Agents: Interactive Simulacra – Park et al. (2023)
- Voyager: An Open-Ended Embodied Agent – Wang et al. (2023)
- AutoGen: Enabling Next-Gen LLM Applications – Wu et al. (2023)
- MetaGPT: Meta Programming for Multi-Agent – Hong et al. (2023)
- CAMEL: Communicative Agents for Mind Exploration – Li et al. (2023)

- HuggingGPT: Solving AI Tasks with ChatGPT – Shen et al. (2023)
- SWE-agent: Agent Computer Interfaces – Yang et al. (2024)
- WebArena: A Realistic Web Environment – Zhou et al. (2023)
- OSWorld: Benchmarking Multimodal Agents – Xie et al. (2024)
- OmniParser for Pure Vision Based GUI Agent – Lu et al. (2024)
- Plan-and-Solve Prompting – Wang et al. (2023)
- CodeAct: Code-based Agent Actions – Wang et al. (2024)
- Tree of Thoughts – Yao et al. (2023)
- Self-Refine: Iterative Refinement – Madaan et al. (2023)
- Prompt Injection Attacks and Defenses – Liu et al. (2023)
- GraphRAG: From Local to Global – Edge et al. (2024)
- RAGAS: Automated Evaluation of RAG – Es et al. (2023)

Final Words

You are building one of the hardest and most impressive projects a software engineer can build. On Day 365, you will have a system that rivals commercial products built by teams of 10–50 engineers. The difference between you and them is that you will understand every single line of code and every architectural decision — because you built it yourself, one day at a time.

Stick to the plan. Ship working code every single day. When you hit a wall (and you will), re-read the relevant section, ask an AI, read the source code. The wall always has a door.

Good luck. Build something legendary. ■